SinkAware: Approximate Low-Rank Fixed-Sink Priors for Attention

Anonymous authors
Paper under double-blind review

Abstract

Attention sinks are fixed early tokens that often receive substantial attention. A tempting systems idea is to precompute or reuse their contribution. We show that exact static reuse is invalid under standard softmax attention because fixed sink-token logits remain query-dependent. We therefore test a narrower approximation: per-head low-rank predictors for fixed sink logits while keeping all non-sink scores exact. On Mac-local distilgpt2 traces, rank-2 prediction reduces aggregate held-out attention-output relative L2 error to 0.141 versus 0.170 for a position-only predictor, while the cost model estimates 0.531x exact four-sink QK multiply-adds. A layer-head paired analysis weakens the claim: rank-2 improves output relative L2 by only 0.0297 ± 0.0378 across 72 layer-head cells and wins $20/72$ cells. A new validation-selected head gate weakens it further: selecting $19/72$ heads on validation raises held-out output relative L2 to 0.204 versus 0.172 for position-only and 0.142 for all-rank2. This is not an end-to-end quality or GPU speed result. It is a bounded correctness and repeatability gate for deciding whether approximate sink prediction is worth implementing.

1 Problem

For query q_t, sink keys K_s, and non-sink keys K_n, the sink logits $q_t K_s^\top$ are still query-dependent. Any exact method that skips them changes the softmax denominator or attention weights. SinkAware kills that exact static-prior branch and revives only an approximate one.

2 Approximate Branch

The live branch predicts fixed sink-token logits from a low-dimensional query feature. Non-sink logits remain exact. The branch is useful only if the approximation is cheap enough and attention-output drift stays bounded. Table 1 reports the original aggregate layer means. This view keeps rank-2 alive: it improves output relative L2 and sink-mass error over position-only while staying under the simple multiply-add estimate for exact four-sink QK.

Predictor	Sink RMSE	Sink-mass MAE	Attention L1	Output rel-L2
static	1229.509	0.095	0.217	0.193
position	1197.390	0.076	0.175	0.170
rank1	1079.541	0.067	0.157	0.160
rank2	1001.506	0.055	0.135	0.141
rank4	838.320	0.045	0.115	0.122
rank8	699.232	0.040	0.104	0.107

Table 1: 48-trace distilgpt2 softmax/output probe. Lower is better.

Table 2 is the reviewer-facing caveat. The same probe now reports layer-head paired improvements over the position-only predictor. Positive values mean lower error than

position-only. Rank-2's mean output improvement is positive, but the interval overlaps zero and the win rate is only 0.278. This makes the branch weakly alive, not established.

Predictor	Output rel-L2 improve	95% CI	Win rate	Sink-mass improve
rank1	0.0257	0.0290	0.250	0.0086
rank2	0.0297	0.0378	0.278	0.0206
rank4	0.0596	0.0465	0.333	0.0308
rank8	0.0955	0.0805	0.347	0.0360

Table 2: Layer-head paired improvements over position-only across 72 layer-head cells. Positive is better; intervals are normal 95% intervals over layer-head cells.

We also tested the obvious Mac-local rescue: select rank-2 only for heads where rank-2 beats position-only on a validation split, then evaluate that mixed policy on held-out tokens. The rule selected 19/72 heads but failed held-out: output relative L2 was 0.204 for validation-selected rank-2, compared with 0.172 for position-only and 0.142 for all-rank2. This rules out simple validation head selection as a pre-GPU rescue.

3 Reference and Kernel Gates

The Phase 3 reference specifies the exact computation to preserve during kernel work. Given query q, keys K, values V, and predicted fixed-sink logits $\hat{s}$, SinkAware computes all tail logits $qK_{tail}^{\top}$ exactly, replaces only the first sink-token logits with $\hat{s}$, and then runs the ordinary softmax denominator over the concatenated vector. Unit tests verify that when $\hat{s}$ equals exact sink logits, the reference reproduces exact attention to numerical tolerance. This keeps the live branch separate from the killed exact-static-reuse branch.

The Phase 4 Triton-interpreter scaffold now targets the same operator for a one-head scalar-output synthetic gate. Triton's official debugging documentation describes interpreter mode as a CPU simulation path enabled by `TRITON_INTERPRET=1`, bypassing compilation for debugging.[1] In the current Mac environment `TRITON_INTERPRET=1` is set for the readiness check, but `triton` is not importable in `venv_arm64` and `torch.cuda.is_available()` is false. The execution tests are therefore written but skip locally, while a non-skipped readiness test records the dependency blocker. This is useful for human review because the kernel correctness contract is explicit before any native timing work, but it is not evidence of GPU performance.

4 Decision

SinkAware is alive only as an all-head approximate low-rank correctness and native-GPU gate. Exact static reuse remains dead, and the simple head-selective rescue is weakened. Promotion now requires a repeatable all-head rank-2 quality result, a better stability mechanism than validation head selection, and native runs that show real speed or memory-traffic advantage over exact attention while still beating the position-only predictor on quality.

5 Artifacts

- `experimental/sinkaware/phase2/real_qk_sink_softmax_output_probe.md`
- `experimental/sinkaware/phase2/head_selective_sink_gate.md`
- `experimental/sinkaware/phase2/qk_sink_cost_model.md`
- `experimental/sinkaware/phase3/approx_sink_attention_reference.md`
- `experimental/sinkaware/phase4/approx_sink_attention_triton_gate.md`

[1] `https://triton-lang.org/main/programming-guide/chapter-3/debugging.html`

- experimental/sinkaware/phase2/gpu_gate_runbook.md
- experimental/sinkaware/paper/reviewer_pack.md